

From LLM to Agent

From LLM to Agent

Building a Clinic Triage Assistant from Scratch

Hi, Ive had headaches for 3 days. I take warfarin daily. Should I be worried? And can I book an appointment with Dr. Cohen next week?

This notebook answers one question: what is an AI agent, and how is it different from an LLM?

The shortest honest answer is a formula:

But a formula doesnt teach anything. So well build an agent - one capability at a time, each motivated by a concrete failure of the previous version. By the end, youll have constructed a working clinic-triage assistant that can read patient records, check drug interactions, look up medical knowledge, book appointments, and double-check its own work - all from a single patient message.

What you'll learn

1. Why an LLM alone cannot solve real-world tasks involving state, current data, or actions.
2. How tool use (function calling) closes the action gap.
3. Why one tool call is not enough, and how the ReAct loop (Thought ? Action ? Observation) is the heart of every agent framework.
4. How memory lets an agent stay coherent across turns.
5. How Retrieval-Augmented Generation (RAG) grounds an agent in a real knowledge base.
6. How self-critique adds a safety net.

Prerequisites

- * Comfortable with Python, dictionaries, loops, and JSON.
- * Basic familiarity with calling an API.
- * An OpenAI API key with access to (cheap; a full run of this notebook costs a few cents).

A warning before we start

This is a teaching toy, not a clinical system. Real healthcare AI must clear bars this notebook does not even attempt: HIPAA, audit logs, bias evaluation, validated medical knowledge bases, human-in-the-loop review, liability engineering, and regulatory approval. Nothing here should be deployed. We use healthcare precisely because the gap between sounds smart and is safe is visible - which is exactly the gap agents have to reckon with.

Why healthcare?

Every pedagogical example involves trade-offs. We picked healthcare because it makes the differences between an LLM and an agent unusually vivid:

Each row of this table maps to a section of the notebook. By the end, every LLM fails on the right will be addressed.

Code

```
# =====
# Setup - install once, then import
# =====
# pip install openai numpy
import os
import json
import numpy as np
from openai import OpenAI
from textwrap import fill
# The Responses API is OpenAI's modern, agent-first endpoint.
# It replaces the older Assistants API and is strictly more powerful
# than the Chat Completions API for tool-use workflows.
import os
from dotenv import load_dotenv
import textwrap
import truststore
truststore.inject_into_ssl()
load_dotenv('/Users/shivam13juna/Documents/scaler/iitr_classes/llm_ref/openai_key.env') #
reads .env file in the current directory
api_key = os.getenv("OPENAI_API_KEY")
if not api_key:
    raise ValueError(
        "OPENAI_API_KEY not found! "
        "Make sure you have a .env file with: OPENAI_API_KEY=sk-..."
    )
client = OpenAI(api_key=api_key) # reads OPENAI_API_KEY from environment
MODEL = "gpt-5-nano" # cheap, fast, good enough for teaching.
# Swap for gpt-5 or gpt-4o in production.
# Sanity check - tiny call to confirm everything is wired up.
resp = client.responses.create(
    model=MODEL,
    input="Say 'ready' and nothing else.",
)
print(resp.output_text)
```

Output

```
ready
```

Part 1 - What an LLM Is, and What It Isn't

Before we can appreciate an agent, we need an honest picture of the thing at its core: the LLM.

1.1 A refresher: what is an LLM, really?

A Large Language Model is a neural network trained on enormous amounts of text to predict the next token. That is all it does. Everything else - question answering, coding, translation - is a downstream consequence of being very good at next-token prediction, plus some fine-tuning to make it behave conversationally.

From this picture, three hard constraints fall out - and these are the constraints that motivate everything in this notebook:

1. Frozen knowledge. The model's parameters were fixed at training time. It does not know what happened yesterday. It does not know your patient record. It does not know the current price of anything. When asked about things it doesn't know, a well-trained LLM should refuse - but in practice, it often confabulates plausibly.
2. Stateless. Each call is independent. The model does not remember your last question unless you re-send it. The model has no self persisting between calls.
3. No hands. The model outputs text. It cannot send an email, update a database, book an appointment, or call a weather API. It can only produce characters.

Keep these three constraints in mind. Every capability we add to our agent is a way of working around one of them.

Code

```
# A minimal LLM call. No tools, no memory, no loop. Just text in, text out.
resp = client.responses.create(
    model=MODEL,
    input="In one sentence: what is the capital of France?",
)
print(resp.output_text)
```

Output

The capital of France is Paris.

1.2 The patient scenario

Imagine you run a small clinic. A patient sends this message through your portal:

Hi, Ive had headaches for 3 days. I take warfarin daily. Should I be worried? And can I book an appointment with Dr. Cohen next week?

Take a moment to list, in your head, everything a human receptionist (or nurse) would actually do to handle this:

- * Look up the patients record (identity, history, current medications).
- * Recognise warfarin as an anticoagulant and think about drug/condition interactions with headache.
- * Know what red flags in headache symptoms warrant escalation.
- * Check Dr. Cohens calendar.
- * Book the appointment.
- * Respond empathetically and clearly.
- * Remember all of this for the patients next message.

Thats at least six distinct capabilities, most of which require external systems, not just knowing stuff. Now lets see what a bare LLM does with this.

Code

```
PATIENT_MESSAGE = (
    "Hi, I've had headaches for 3 days. I take warfarin daily. "
    "Should I be worried? And can I book an appointment with Dr. Cohen next week?"
)
resp = client.responses.create(
    model=MODEL,
    input=PATIENT_MESSAGE,
    instructions=(
        "You are a helpful clinic assistant. Respond to the patient's message."
    ),
)
print(fill(resp.output_text, width=88))
```

Output

I'm glad you asked. A new or worsening headache on warfarin can be more serious, so it's important to evaluate carefully. Red flags that require urgent care now - Severe or "worst headache of your life" - Headache with neck stiffness, fever, confusion, or new seizures - weakness, numbness, or slurred speech on one side - Vision changes, severe vomiting, or you've had head trauma - If you know your INR is high or you notice bleeding (unusual bruising, blood in urine/stools) What to do today - If you have any red flags, or you're unsure how to proceed, seek urgent medical attention (emergency department or urgent care). - Do not stop or adjust your warfarin dose without speaking to a clinician. If you have an INR result, tell the clinician when you get evaluated. - Avoid NSAIDs (like ibuprofen or naproxen) while on warfarin unless advised by your doctor. Acetaminophen is generally safer for pain relief, but check with your clinician first. Booking an appointment with Dr. Cohen next week I can help check Dr. Cohen's availability and arrange a slot for you. To speed things up, please tell me: - Do you prefer in-person or telemedicine next week? - Which day/time window works best? - Your full name, date of birth, and the best contact number or email - Do you know your latest INR value (if yes, please share)? - Any other symptoms you've noticed (nausea/vomiting, photophobia, etc.) If you'd prefer, you can also call the clinic directly or use the online booking form with these details. Your safety is the priority. If you have any red flags now, please seek urgent care right away. If you'd like, tell me your preferences and I'll start checking Dr. Cohen's availability.

1.3 Deconstructing the failure

The LLMs response probably sounded reasonable - thats the trap. Lets pick it apart. One or more of the following will almost certainly be true:

This is not a bad model. This is an LLM being asked to do things an LLM fundamentally cannot do.

The action gap: LLMs generate; agents do. Everything we build from here on is about crossing that gap.

? Quiz - Part 1

Q1. Which of the following is NOT a hard constraint of a bare LLM?

- (a) It cannot access information after its training cutoff.
- (b) It cannot remember prior conversations unless they are re-sent to it.
- (c) It cannot take actions in the external world.
- (d) It cannot generate grammatical English.

Q2. The patient asks the LLM to book an appointment. The LLM replies: Ive booked you in with Dr. Cohen on Thursday at 2pm. What has actually happened?

Q3. In what sense is confabulation (confidently making things up) a predictable behaviour of LLMs rather than a bug?

Part 2 - Capability 1: Tools (Function Calling)

Lets cross the action gap.

2.1 The idea: let the LLM call your code

Here is the central trick, and it is simpler than it sounds.

We give the LLM a menu of functions we have written. We describe each one - name, what it does, what arguments it takes - in a structured format (JSON schema). When the LLM gets a user question, it has two options:

1. Answer in plain text, as before.
2. Output a request to call one of the functions, with specific arguments.

If the LLM chooses option 2, we execute the function on our side and feed the result back to the LLM. The LLM then uses that result to compose its final answer.

Crucially: the LLM never runs code. It only requests calls. We retain full control.

This one mechanism - often called tool use or function calling - is the foundation of every agent system. Everything else in this notebook is layered on top.

2.2 Our first tool: drug interaction checker

Lets build a real (tiny) tool. Well hand-code a dictionary of known interactions. In production this would be a call to a licensed drug database; for teaching, a Python dict makes the mechanics crystal clear.

Code

```

# =====
# Tool 1: check_drug_interactions
# =====
# A real implementation would query a database like DrugBank.
# We hardcode a small, obviously-incomplete dict. The point is to
# show the MECHANICS of tool use - not to build a drug database.
_DRUG_INTERACTIONS = {
    frozenset(["warfarin", "ibuprofen"]): {
        "severity": "high",
        "note": "NSAIDs increase bleeding risk when combined with warfarin.",
    },
    frozenset(["warfarin", "aspirin"]): {
        "severity": "high",
        "note": "Both affect clotting; combined bleeding risk is significant.",
    },
    frozenset(["warfarin", "paracetamol"]): {
        "severity": "low",
        "note": "Generally safe at standard doses; monitor INR with chronic use.",
    },
    frozenset(["metformin", "alcohol"]): {
        "severity": "moderate",
        "note": "Increased risk of lactic acidosis.",
    },
}

def check_drug_interactions(drugs: list[str]) -> dict:
    '''Look up interactions between a list of drugs.

    Args:
    drugs: list of drug names, lowercase.

    Returns:
    dict with a list of known interactions, or an empty list.
    '''
    drugs = [d.lower().strip() for d in drugs]
    found = []
    for i in range(len(drugs)):
        for j in range(i + 1, len(drugs)):
            key = frozenset([drugs[i], drugs[j]])
            if key in _DRUG_INTERACTIONS:
                entry = _DRUG_INTERACTIONS[key]
                found.append({
                    "drug_a": drugs[i],
                    "drug_b": drugs[j],
                    "severity": entry["severity"],
                    "note": entry["note"],
                })
    return {"interactions": found, "drugs_checked": drugs}

# Quick smoke test - does our tool work in isolation?
print(check_drug_interactions(["warfarin", "ibuprofen"]))

```

Output

```
{'interactions': [{ 'drug_a': 'warfarin', 'drug_b': 'ibuprofen', 'severity': 'high', 'note':
```


'NSAIDs increase bleeding risk when combined with warfarin.']], 'drugs_checked': ['warfarin',

```
'ibuprofen']}]}
```

2.3 Describing the tool to the LLM

The LLM cannot read our Python code. We have to describe the function to it in a format it understands: a JSON schema. This description tells the LLM three things:

- * Name - what to call the function.
- * Description - when to call it (the LLM uses this to decide).
- * Parameters - what arguments, and of what type.

In the Responses API, this description is a flat dict with .

Code

```
# The tool schema. This is what the LLM sees.
# Notice how much it looks like documentation - because that's what it is,
# written for a reader whose attention span is a prompt.
check_drug_interactions_schema = {
    "type": "function",
    "name": "check_drug_interactions",
    "description": (
        "Check for known interactions between a list of drugs. "
        "Use this whenever a patient mentions multiple medications, or "
        "mentions a new symptom while on an existing medication."
    ),
    "parameters": {
        "type": "object",
        "properties": {
            "drugs": {
                "type": "array",
                "items": {"type": "string"},
                "description": "List of drug names (generic, lowercase).",
            },
        },
        "required": ["drugs"],
        "additionalProperties": False,
    },
}
```

2.4 Calling the LLM with the tool attached

Now we make a Responses API call with . Watch what comes back.

Code

```
PATIENT_MESSAGE = (
    "Hi, I've had headaches for 3 days. I take warfarin and aspirin daily. "
    "Should I be worried? And can I book an appointment with Dr. Cohen next week?"
)
```

Code

```
resp = client.responses.create(
    model=MODEL,
    input=PATIENT_MESSAGE,
    instructions=(
        "You are a clinic assistant. If the patient mentions medications "
        "and symptoms, use your tools to check for relevant interactions."
    ),
    tools=[check_drug_interactions_schema],
)
# response.output is a LIST of items. Each item has a type.
# Types we care about: 'message' (plain text) and 'function_call' (tool request).
for item in resp.output:
    print("---", item.type, "---")
    print(item)
    print()
```

Output

```
--- reasoning ---
ResponseReasoningItem(id='rs_06a6dd19235ce75a0069e0d49333a4819b8b5f5245a26dd172', summary=[],
type='reasoning', content=None, encrypted_content=None, status=None)
--- function_call ---
ResponseFunctionToolCall(arguments='{"drugs":["warfarin","aspirin"]}',
```

```
call_id='call_Htw6dUh5awbs8FH3uRdGC3Nl', name='check_drug_interactions', type='function_call',
id='fc_06a6dd19235ce75a0069e0d49c594c819b906e3e415018878b', namespace=None,
status='completed')
```

2.5 What just happened

If the LLM decided to call the tool, you'll see an item with `tool_calls` containing:

- * - the function it wants to call `()`.
- * - a string containing JSON, e.g. `{}.`
- * - a unique identifier we'll need to match the output back to this call.

The LLM did not execute anything. It produced a request for us to execute. This is the hand-off. Our job now:

1. Parse the JSON.
2. Run the actual Python function.
3. Send the result back to the LLM in a follow-up API call, tagged with the same `call_id`.
4. Receive the LLM's final human-facing answer.

Code

```
# Step 1 - find the function_call item
tool_call = next(item for item in resp.output if item.type == "function_call")
print("Calling:", tool_call.name)
print("With args:", tool_call.arguments)
print("Call ID:", tool_call.call_id)
# Step 2 - parse args and execute
args = json.loads(tool_call.arguments)
result = check_drug_interactions(**args)
print("\nResult of running the tool locally:")
print(result)
```

Output

```
Calling: check_drug_interactions
With args: {"drugs":["warfarin","aspirin"]}
Call ID: call_Htw6dUh5aWbs8FH3uRdGC3Nl
Result of running the tool locally:
{'interactions': [{'drug_a': 'warfarin', 'drug_b': 'aspirin', 'severity': 'high', 'note':
'Both affect clotting; combined bleeding risk is significant.'}], 'drugs_checked':
```

```
['warfarin', 'aspirin']}]
```

Code

```
# Step 3 - send the result back to the LLM.
# The Responses API is stateless by default: to continue the "conversation",
# we re-send the history. That history = original input + the model's outputs
# + our new function_call_output item.
#
# Note: we pass `resp.output` items through directly (not .model_dump()'d).
# The SDK knows how to serialise them. If we .model_dump()'d them we'd leak
# server-side fields like `status` that the API rejects on input - a common
# first-time footgun with the Responses API.
followup_input = [
    {"role": "user", "content": PATIENT_MESSAGE},
    # The model's output items, echoed back verbatim.
    *resp.output,
    # Our response to the tool call. Note the call_id links them.
    {
        "type": "function_call_output",
        "call_id": tool_call.call_id,
        "output": json.dumps(result),
    },
]
resp2 = client.responses.create(
    model=MODEL,
    input=followup_input,
    instructions=(
        "You are a clinic assistant. If the patient mentions medications "
        "and symptoms, use your tools to check for relevant interactions."
    ),
    tools=[check_drug_interactions_schema],
)
print(fill(resp2.output_text, width=88))
```

Output

Thanks for checking in. Here's what I can tell you based on your meds and symptoms:
Drug interaction note - Warfarin and aspirin together create a high risk of bleeding. This combination should be monitored closely, and you should be cautious about any new or worsening symptoms. Should you be worried? - A new, persistent headache (3 days) while on blood thinners can be a red flag for serious issues (including bleeding in the brain or other sites). It's not something to ignore. - Immediate red-flag symptoms to seek urgent care/ER right away: - Sudden "thunderclap" headache or very severe head pain - New weakness, numbness, or facial droop on one side - Trouble speaking, confusion, or vision changes - Neck stiffness, fever, or vomiting that won't stop - Head injury, or you're noticing easy bruising or bleeding (nosebleeds, gums) that is new or worsening - If you have none of these red flags, you should still arrange a timely medical evaluation because of the bleeding risk with your meds. What to do now - Do not stop or change warfarin or aspirin on your own. If you're worried about bleeding or your symptoms worsen, contact your clinician or go to urgent care/ER. - If you'd like, I can help you set up an appointment with Dr. Cohen for next week. To do that, please provide: - Your full name and date of birth (to verify your chart) - Best contact phone number - Preferred day/time window next week - In-person or telemedicine visit Would you like me to check Dr. Cohen's availability for next week and propose some slots, then help place a booking? If yes, share the details above and your preferred format (in-person or telemedicine).

Code


```

resp3 = client.responses.create(
model=MODEL,
previous_response_id=resp.id,          # ? server remembers the history
input=[{                               # ? you only send the NEW stuff
"type": "function_call_output",
"call_id": tool_call.call_id,
"output": json.dumps(result),
}],
instructions=(
"You are a clinic assistant. If the patient mentions medications "
"and symptoms, use your tools to check for relevant interactions."
),
tools=[check_drug_interactions_schema],
)
print(fill(resp3.output_text, width=88))

```

Output

Thanks for checking in. I checked for interactions between your meds: - Warfarin and aspirin together: high risk. Both affect clotting, and using them together significantly increases bleeding risk. What this means for your headaches: - A new or persistent headache (like 3 days) can be concerning when you're on anticoagulants/antiplatelets. It could be something minor, but it can also be a signal of a bleed in or around the brain, especially if it's sudden, severe, or accompanied by other symptoms. - Do not stop either medication on your own. Contact the clinician who prescribes them before making any changes. Seek urgent care now or go to the emergency department if you have any red flags, such as: - Sudden, severe or "thunderclap" headache - Confusion, weakness or numbness on one side, trouble speaking - Severe neck stiffness, fever, vomiting - Black, tarry stools or vomiting blood - Head injury with worsening symptoms - Excessive or easy bruising, unusual bleeding gums or nosebleeds If you're not experiencing red flags, you should still be evaluated soon due to your meds. I can help arrange an appointment with Dr. Cohen next week. To book: - Tell me your preferred day and time next week (and whether you want in-person or virtual). - Please provide your full name and the best contact number. - If you have an ID or insurance details the clinic uses, share those too. Optional, for the appointment: - Do you want a quick check of your latest INR value and any recent lab results reviewed during the visit? If you'd prefer to cancel or delay, or if you're worried about red-flag symptoms right now, let me know and I'll guide you accordingly.

2.6 Reflect on what changed

Compare this answer to Part 1s LLM-alone answer. The tool-using version, when it talks about warfarin and NSAIDs, is quoting our database - not inventing from pre-training. We have closed one small piece of the action gap: the look something up kind.

But notice the limits:

1. One tool, one call. The model only used the interaction checker. It still hand-waved patient records, symptom severity, and booking.
2. No sequence. The model decided what to do in a single shot. Real tasks need the model to react to what it finds and decide what to do next.

Both of these are fixed by the same thing: making tool use a loop.

? Quiz - Part 2

Q1. In tool-use, who actually runs the Python code?

Q2. What is `for`, and what would break if we didnt send it back?

Q3. Our `function` has a hardcoded dict of 4 entries. Why might it be better, in a tutorial, to keep the dataset tiny rather than plug in a real drug database?

Part 3 - Capability 2: The Loop (ReAct)

A single tool call is useful. But the patients message needs several tools, used in sequence, with each step informing the next. That pattern has a name: ReAct.

3.1 Why one call is not enough

Lets re-read the patients message:

Hi, Ive had headaches for 3 days. I take warfarin daily. Should I be worried? And can I book an appointment with Dr. Cohen next week?

To properly handle this, an agent needs to:

1. Look up who this patient is (retrieve their record).
2. Given the record + current meds + symptoms, check relevant interactions.
3. Optionally look up medical knowledge about headaches lasting 3 days on warfarin.
4. Check Dr. Cohens availability.
5. Book the appointment.
6. Compose a reply.

Each step depends on the one before it. Step 2 needs the meds from step 1. Step 5 needs the slot from step 4. No single API call can do this - the model doesn't know what's in the patient record until it asks.

We need a loop.

3.2 The ReAct pattern

ReAct (Reasoning + Acting, from a 2022 paper by Yao et al.) is the dominant agent architecture. It is not a library - it is a pattern you implement in a few lines of code:

- * Thought: the LLM decides what to do next (implicit in modern tool-using models - they don't always verbalise the thought, but they're doing it internally).

- * Action: the LLM emits a tool call, or a final answer.

- * Observation: we execute the tool and feed the result back.

Then we loop. The LLM sees the accumulating history (original question + all prior tool calls + all prior results) and decides the next action. It stops when it has enough to answer in plain text.

Everything we've been building is a ReAct loop with . Let's let it iterate.

3.3 More tools

To make the loop interesting, we need more tools. Below are three more, each with a tiny mock backend. Read the data closely - the patient's record references , so the drug-interaction tool from Part 2 will matter inside the loop.

Code

```

# =====
# Tool 2: get_patient_record
# =====
_PATIENT_DB = {
    "P001": {
        "patient_id": "P001",
        "name": "Rivka Weiss",
        "age": 68,
        "medications": ["warfarin", "lisinopril"],
        "conditions": ["atrial fibrillation", "hypertension"],
        "last_visit": "2026-03-12",
        "notes": "INR stable on 5mg warfarin daily.",
    },
    "P002": {
        "patient_id": "P002",
        "name": "Yonatan Cohen",
        "age": 34,
        "medications": ["metformin"],
        "conditions": ["type 2 diabetes"],
        "last_visit": "2026-02-02",
        "notes": "HbA1c trending down.",
    },
}

def get_patient_record(patient_id: str) -> dict:
    '''Retrieve a patient record by ID.'''
    record = _PATIENT_DB.get(patient_id)
    if record is None:
        return {"error": f"No patient found with id {patient_id}"}
    return record

```

Code

```
# =====
# Tool 3: book_appointment
# =====
# A *side-effectful* tool. In reality this would hit a calendar API.
_BOOKED_APPOINTMENTS = []
_DOCTOR_AVAILABILITY = {
    "Dr. Cohen": ["2026-04-20 09:00", "2026-04-20 14:00", "2026-04-22 10:30"],
    "Dr. Levi": ["2026-04-21 11:00", "2026-04-23 09:30"],
}
def book_appointment(patient_id: str, doctor: str, datetime_iso: str) -> dict:
    '''Book an appointment. Fails if the slot isn't available.'''
    available = _DOCTOR_AVAILABILITY.get(doctor, [])
    if datetime_iso not in available:
        return {
            "success": False,
            "reason": "Slot not available.",
            "available_slots": available,
        }
    _DOCTOR_AVAILABILITY[doctor].remove(datetime_iso)
    confirmation = f"APPT-{{len(_BOOKED_APPOINTMENTS) + 1001}}"
    _BOOKED_APPOINTMENTS.append({
        "confirmation": confirmation,
        "patient_id": patient_id,
        "doctor": doctor,
        "datetime": datetime_iso,
    })
    return {"success": True, "confirmation": confirmation}
```

Code

```
# =====  
# Tool 4: get_doctor_availability  
# =====  
def get_doctor_availability(doctor: str) -> dict:  
    '''List free slots for a doctor.'''  
    slots = _DOCTOR_AVAILABILITY.get(doctor)  
    if slots is None:  
        return {"error": f"Unknown doctor: {doctor}"}  
    return {"doctor": doctor, "available_slots": slots}
```

Code

```

# Schemas for the three new tools. Same flat format as before.
get_patient_record_schema = {
    "type": "function",
    "name": "get_patient_record",
    "description": (
        "Retrieve a patient's medical record by ID. Use this early whenever "
        "you need to know the patient's medications, conditions, or history."
    ),
    "parameters": {
        "type": "object",
        "properties": {
            "patient_id": {"type": "string", "description": "The patient ID, e.g. P001."},
        },
        "required": ["patient_id"],
        "additionalProperties": False,
    },
}

get_doctor_availability_schema = {
    "type": "function",
    "name": "get_doctor_availability",
    "description": "List available appointment slots for a given doctor.",
    "parameters": {
        "type": "object",
        "properties": {
            "doctor": {"type": "string", "description": "Doctor's name, e.g. 'Dr. Cohen'."},
        },
        "required": ["doctor"],
        "additionalProperties": False,
    },
}

book_appointment_schema = {
    "type": "function",
    "name": "book_appointment",
    "description": (
        "Book an appointment for a patient. Use AFTER confirming the slot is "
        "available via get_doctor_availability."
    ),
    "parameters": {
        "type": "object",
        "properties": {
            "patient_id": {"type": "string"},
            "doctor": {"type": "string"},
            "datetime_iso": {
                "type": "string",
                "description": "ISO-format datetime, e.g. '2026-04-20 09:00'.",
            },
        },
        "required": ["patient_id", "doctor", "datetime_iso"],
        "additionalProperties": False,
    },
}

# A registry lets us look up the Python implementation by the name
# the LLM used in its tool call.
TOOL_IMPLEMENTATIONS = {
    "check_drug_interactions": check_drug_interactions,
    "get_patient_record": get_patient_record,
    "get_doctor_availability": get_doctor_availability,
    "book_appointment": book_appointment,
}

TOOL_SCHEMAS = [
    check_drug_interactions_schema,
    get_patient_record_schema,
    get_doctor_availability_schema,

```

3.4 The loop itself

This is the single most important code cell in the notebook. Read it twice. It is ~30 lines of Python, and it is the core of every agent framework you have ever heard of.

The loop has four parts:

1. Start with a message history containing just the users question.
2. Call the LLM with the history and the tool schemas.
3. If the LLM asked for tool calls: execute them, append results to history, loop.
4. If the LLM gave a plain text answer (no tool calls): were done, return it.

We cap iterations as a safety measure - a bug in the prompt or a model quirk can cause runaway loops.

Code


```

SYSTEM_PROMPT = '''You are a clinic triage assistant. You have tools to:
- look up patient records,
- check drug interactions,
- check doctor availability,
- book appointments.
Rules:
- Always retrieve the patient record before giving medical advice.
- Check drug interactions when the patient mentions symptoms while on medication.
- Before booking, confirm the slot is available.
- Be empathetic and plain-spoken. Flag anything that warrants urgent human review.
- If you do not know the patient_id, ask the user for it - do NOT guess.
'''

def run_agent(user_message, message_history=None, max_iterations=8, verbose=True):
    '''The ReAct loop, from scratch.
    Returns (final_text, full_message_history).
    '''
    # Build or extend the conversation history.
    if message_history is None:
        message_history = []
    message_history = message_history + [
        {"role": "user", "content": user_message}
    ]
    for iteration in range(max_iterations):
        if verbose:
            print(f"\n???????? Iteration {iteration + 1} ????????")
        # --- THINK+ACT: ask the LLM what to do next ---
        resp = client.responses.create(
            model=MODEL,
            input=message_history,
            instructions=SYSTEM_PROMPT,
            tools=TOOL_SCHEMAS,
        )
        # Append every output item to history so the model sees its own work.
        # Note: append the raw SDK objects, not .model_dump()'d dicts - the
        # latter leaks server-side fields (like `status`) that the API rejects
        # when echoed back as input.
        message_history.extend(resp.output)
        # Did the model emit any tool calls?
        tool_calls = [item for item in resp.output if item.type == "function_call"]
        if not tool_calls:
            # No tool calls ? the model produced a final answer.
            if verbose:
                print("? Final answer.")
            return resp.output_text, message_history
        # --- OBSERVE: run each requested tool, feed back results ---
        for tc in tool_calls:
            name = tc.name
            args = json.loads(tc.arguments)
            if verbose:
                print(f" ? tool: {name}({args})")
            impl = TOOL_IMPLEMENTATIONS.get(name)
            if impl is None:
                result = {"error": f"Unknown tool: {name}"}
            else:
                try:
                    result = impl(**args)
                except Exception as e:
                    result = {"error": f"Tool raised: {type(e).__name__}: {e}"}
            if verbose:
                short = str(result)
                print(f" ? result: {short[:120]}{'...' if len(short) > 120 else ''}")
            message_history.append({
                "type": "function_call_output",

```

3.5 Run the scenario through the agent

We'll give the agent the same patient message - but this time, we also tell it the patient's ID so it knows where to start looking. Watch the trace carefully: you should see the agent picking tools in a sensible order.

Code

```
final_answer, history = run_agent(
    "Patient P001 says: " + PATIENT_MESSAGE + " They are asking for any slot."
)
print("\n\n???????????? FINAL ANSWER ??????????")
print(fill(final_answer, width=88))
```

Output

```
???????? Iteration 1 ????????
? tool: get_patient_record({'patient_id': 'P001'})
? result: {'patient_id': 'P001', 'name': 'Rivka Weiss', 'age': 68, 'medications': ['warfarin',
'lisinopril'], 'conditions': ['atri...
? tool: get_doctor_availability({'doctor': 'Dr. Cohen'})
? result: {'doctor': 'Dr. Cohen', 'available_slots': ['2026-04-20 09:00', '2026-04-20 14:00',
'2026-04-22 10:30']}
???????? Iteration 2 ????????
? Final answer.
???????????? FINAL ANSWER ??????????
I'm glad you reached out. I pulled up your chart and here's what I found and what I
recommend. What I found in your record - Patient: Rivka Weiss, age 68 - Medications
listed: warfarin and lisinopril - Note: INR has been stable on 5 mg warfarin daily -
Aspirin is not listed in your chart. If you are taking aspirin daily in addition to
warfarin, please tell me right away. Warfarin plus aspirin increases bleeding risk and
needs careful evaluation. Headache and anticoagulation: red flags to watch for. Seek
urgent care now if you have any of these: - Sudden, severe, "worst headache" or
"thunderclap" headache - Numbness/weakness, trouble speaking or confusion - Changes in
vision, severe neck stiffness, fever, or new vomiting - Unusual bleeding (easy bruising,
blood in stool or urine, prolonged bleeding from cuts). If you don't have red flags, you
still should be evaluated for persistent headaches while on anticoagulants, to rule out
serious causes and to review your meds and INR. Appointment availability with Dr. Cohen
(next week) Available slots: - 2026-04-20 at 09:00 - 2026-04-20 at 14:00 - 2026-04-22 at
10:30. Please tell me which slot you'd like to book (or suggest another time), and I'll
book it for you after confirming it's still open. If you'd like, I can also arrange a
quick INR check around the appointment or coordinate a same-day triage if your symptoms
worsen.
```

3.6 Compare to Part 1

Look at the trace above and compare it to the single-shot LLM response from Part 1.

- * The agent retrieved the real record for P001, so it didn't invent history.
- * It checked real drug interactions against real medications, so its bleeding-risk note is grounded.
- * It inspected real availability, so any appointment it proposed actually exists.
- * If it booked an appointment, it actually mutated our (mock) calendar - the appointment really is in .

What the LLM is doing internally is unchanged: it is still just predicting tokens. What changed is the scaffolding around the LLM - the loop, the tools, the history threading. That scaffolding is the agent.

Code

```
# Let's verify the side effect - the agent actually changed the world.
print("Booked appointments:", _BOOKED_APPOINTMENTS)
print("\nDr. Cohen's remaining slots:", _DOCTOR_AVAILABILITY["Dr. Cohen"])
```

Output

```
Booked appointments: []
Dr. Cohen's remaining slots: ['2026-04-20 09:00', '2026-04-20 14:00', '2026-04-22 10:30']
```

3.7 Guardrails: why `max\_iterations` is non-negotiable

An agent loop can - and occasionally will - misbehave:

- * The model might keep re-calling the same tool with the same arguments.
- * It might pursue a dead-end reasoning chain indefinitely.
- * A prompt bug can make it never decide its done.

Every line of this loop that talks to the API costs money, takes time, and risks side effects. A hard iteration cap is not a nice-to-have - it is a requirement. Alongside it, production systems add:

- * Step budgets (total tokens across the loop).
- * Duplicate-call detection (refuse to call the same tool with the same args twice).
- * Tool-level quotas (max 3 bookings per run).
- * Logging / tracing (every step persisted for audit).

We'll keep just in this notebook, but know that the rest of the list exists.

? Quiz - Part 3

Q1. Why does the agent's final answer tend to be more trustworthy than the bare LLMs, even though both are produced by the same model?

Q2. What is the difference between a `message` item and a `history` item in our message history?

Q3. Suppose we set `max_message_history`. Which features of the agent survive? Which break?

Part 4 - Capability 3: Memory

Our agent is stateless per call. Let's fix that.

4.1 Two kinds of memory

Humans have two kinds of memory that matter for conversations:

- * Short-term - what we said a minute ago. In agent terms: the current conversation buffer.

- * Long-term - facts about the world, about us, about this patient. In agent terms: a database, a file, a vector store.

Our tools already provide long-term memory via `vector_store` - the record persists across runs because it lives in `vector_store`. That's fine. What we don't yet have is short-term memory: if the patient sends a second message, the agent doesn't remember the first one.

That's just a matter of threading the `history` between turns.

Code

```
# Turn 2 - the patient replies. We pass the accumulated history back in.
answer2, history = run_agent(
    "Actually, can we make that Wednesday instead? Same week.",
    message_history=history
)
print("\nAGENT (turn 2):")
print(fill(answer2, width=88))
```

Output

```
???????? Iteration 1 ????????
? Final answer.
AGENT (turn 2):
Thanks for the update. Here's what I can do: - Wednesday slot: 2026-04-22 at 10:30 is
currently available with Dr. Cohen. Would you like me to book that appointment for you?
A quick safety note before we book - Your chart shows warfarin and lisinopril; aspirin
usage isn't listed. If you are taking aspirin daily in addition to warfarin, that
increases bleeding risk. Please confirm whether you're taking aspirin daily so I can
tailor the plan and note it for the clinician. - If you have any red flags (sudden
severe headache, weakness/numbness, trouble speaking, vision changes, high fever,
uncontrolled bleeding), seek urgent care now. Optional: I can arrange an INR check
around the visit or set up a same-week triage if your symptoms worsen. Let me know if
you'd like me to include either of these.
```

Code

```
# Turn 2 - the patient replies. We pass the accumulated history back in.
answer3, history = run_agent(
    "Sure",
    message_history=history
)
print("\nAGENT (turn 3):")
print(fill(answer3, width=88))
```

Output

```
???????? Iteration 1 ????????
? tool: book_appointment({'patient_id': 'P001', 'doctor': 'Dr. Cohen', 'datetime_iso': '2026-04-22 10:30'})
? result: {'success': True, 'confirmation': 'APPT-1001'}
???????? Iteration 2 ????????
? Final answer.
AGENT (turn 3):
You're all set. I've booked your appointment with Dr. Cohen for Wednesday, 2026-04-22 at 10:30. Booking details: - Appointment: Dr. Cohen, 2026-04-22 10:30 - Confirmation code: APPT-1001 Important reminders: - Your chart lists warfarin and lisinopril; aspirin usage isn't documented. If you're taking aspirin daily, please tell me or bring the medication list to the appointment. Combining warfarin with aspirin can increase bleeding risk. - Bring or be ready to share a current list of all meds, including over-the-counter and supplements. - If you have warning signs (sudden severe headache, weakness or numbness, confusion, trouble speaking, vision changes, fever with neck stiffness, or uncontrolled bleeding), seek urgent care immediately. - If you'd like, I can arrange an INR check around the visit or set up a short triage if your symptoms worsen before then. Let me know if you want me to add either option. Would you like me to note in your appointment that you may be on aspirin, and to remind the clinic to check your INR? I can also set a reminder for you closer to the day.
```

4.2 What the agent "remembers"

The agent has no persistent self. What feels like memory is just the list we pass back in. That list contains:

- * The original user message.
- * Every tool call the agent made last turn.
- * Every tool result it saw last turn.
- * The final answer it gave last turn.
- * The new user message.

When the model sees all of this as context, it can reason about the new message in light of the prior turn - rescheduling the already-booked appointment rather than booking a second one.

This is a crucial piece of intuition: memory in an LLM-based agent is just context. It is not a separate component; it is a discipline of what you put in the prompt.

4.3 The long-term side

A full agent has persistent memory too - facts that survive process restarts. Common implementations:

- * A database (SQL, key-value, graph).
- * A vector store (for semantic retrieval - this is what RAG, next section, uses).
- * A scratchpad file that the agent can read and append to via a tool.

In our clinic example, is the persistent long-term memory. In a production system, would be too. The agent doesn't need to know the storage details - it just needs tools that hide them.

4.4 Context-window limits - the silent ceiling

Short-term memory is bounded by the model's context window. For that's substantial, but not infinite. For very long conversations, production agents typically:

1. Summarise old turns and drop the raw history.
2. Offload old turns to a vector store and retrieve the relevant ones when needed.
3. Run a dedicated memory tool that lets the agent save and look up its own notes.

We won't implement these here, but they use exactly the tools + loop + retrieval machinery we're already building.

? Quiz - Part 4

Q1. Our agent's memory is the list. What does that tell you about where it lives and how it can be lost?

Q2. Why isn't short-term memory a new capability - why is it just threading history?

Q3. If we never trimmed or summarised history, what would eventually go wrong?

Part 5 - Capability 4: Retrieval-Augmented Generation (RAG)

Structured tools handle structured data (drug interactions, records, calendars). But clinical knowledge is mostly unstructured prose - guidelines, red flags, research summaries. For that, we need RAG.

5.1 The gap RAG fills

Ask our agent: Should I be worried about 3-day headaches while on warfarin? - and it will try to answer from the models pre-training. That's exactly the hallucination risk we've been trying to avoid. But we can't put medical guidelines into a structured tool; there are too many, and they are written in prose.

The solution is retrieval. We:

1. Build a corpus of trusted short passages.
2. Convert each passage to an embedding - a dense vector that captures meaning.
3. When the agent wants medical knowledge, embed its query the same way and pull the most similar passages.
4. Give those passages to the model as context, and let it compose its answer from them.

The machinery is not magic. It is three ideas: embeddings, similarity, and pull into the prompt.

5.2 Building the (dummy) corpus

In a real clinic this would be curated excerpts from UpToDate, NICE guidelines, WHO protocols, etc. For teaching we hand-write 8 short passages. Keep them short - we want to see retrieval happen clearly, not drown in prose.

Code


```
# =====
# A tiny medical knowledge base - 8 passages.
# Each is ~1-2 sentences. Real corpora are millions of passages.
# =====
MEDICAL_KB = [
    "Headaches lasting more than 72 hours, especially with sudden onset, may indicate "
    "serious conditions such as subarachnoid haemorrhage and warrant urgent evaluation.",
    "Warfarin is an anticoagulant. Patients on warfarin have increased bleeding risk "
    "and should report unusual bruising, blood in urine or stool, or severe headache.",
    "Red-flag symptoms with headache include sudden severe onset ('thunderclap'), "
    "fever with neck stiffness, new neurological deficits, or head injury.",
    "Combining warfarin with NSAIDs such as ibuprofen or naproxen significantly "
    "increases the risk of gastrointestinal bleeding.",
    "Paracetamol (acetaminophen) is generally the preferred analgesic for patients on "
    "warfarin, at doses up to 3g per day in adults with normal hepatic function.",
    "Tension-type headache is the most common primary headache; typical features include "
    "bilateral, pressing, mild-to-moderate pain without nausea.",
    "Patients on long-term warfarin should have their INR monitored regularly. A target "
    "INR of 2.0 to 3.0 is typical for most indications.",
    "Any headache in a patient on anticoagulation that is severe, sudden, or associated "
    "with altered mental status should be treated as a potential intracranial bleed "
    "until proven otherwise.",
]
```

5.3 Embeddings: turning text into vectors

An embedding is a list of numbers - typically a few hundred to a few thousand dimensions - that represents the meaning of a piece of text. Two texts with similar meaning will have similar embedding vectors.

We don't compute embeddings ourselves; we call OpenAI's embeddings endpoint and get vectors back. We store them alongside the original text.

Code

```
# Embed every passage. One API call, all passages at once.
EMBED_MODEL = "text-embedding-3-small"
emb_response = client.embeddings.create(
    model=EMBED_MODEL,
    input=MEDICAL_KB,
)
# Store (text, vector) pairs. In production this would be a vector DB;
# for us, a list of tuples is more than enough.
KB_EMBEDDINGS = [
    (passage, np.array(datum.embedding, dtype=np.float32))
    for passage, datum in zip(MEDICAL_KB, emb_response.data)
]
print(f"Embedded {len(KB_EMBEDDINGS)} passages.")
print(f"Each vector has {len(KB_EMBEDDINGS[0][1])} dimensions.")
print(f"\nFirst 8 numbers of the first vector:\n{KB_EMBEDDINGS[0][1][:8]}")
```

Output

```
Embedded 8 passages.  
Each vector has 1536 dimensions.  
First 8 numbers of the first vector:  
[-0.0128479  0.02452087  0.06005859  0.00239754 -0.0401001  0.04373169  
0.05206299 -0.00767517]
```

5.4 Similarity search: pure NumPy, no vector DB

Similar vectors mean similar text. The standard way to quantify this is cosine similarity - the cosine of the angle between two vectors. Values near 1 mean very similar; near 0 means unrelated; near -1 means opposite (rare for embeddings).

Cosine similarity of two vectors a and b :

In NumPy, that's three lines.

Code

```
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:  
    '''Cosine similarity between two 1-D vectors.'''  
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))  
def retrieve(query: str, k: int = 3) -> list[dict]:  
    '''Return the top-k passages most similar to the query.'''  
    q_vec = np.array(  
        client.embeddings.create(model=EMBED_MODEL, input=query).data[0].embedding,  
        dtype=np.float32,  
    )  
    scored = [  
        (cosine_similarity(q_vec, vec), passage)  
        for passage, vec in KB_EMBEDDINGS  
    ]  
    scored.sort(key=lambda x: x[0], reverse=True)  
    return [  
        {"score": round(score, 3), "passage": passage}  
        for score, passage in scored[:k]  
    ]  
    # Quick test - does it retrieve sensibly?  
    for hit in retrieve("3-day headache on warfarin, when to worry", k=3):  
        print(f"[score={hit['score']}] {hit['passage']}\n")
```

Output

```
[score=0.59] Any headache in a patient on anticoagulation that is severe, sudden, or associated with altered mental status should be treated as a potential intracranial bleed until proven otherwise.  
[score=0.584] Warfarin is an anticoagulant. Patients on warfarin have increased bleeding risk and should report unusual bruising, blood in urine or stool, or severe headache.
```

5.5 Wrap retrieval as a tool

Now the magic: we turn into a tool the agent can call. From the agent's perspective, RAG is just another function on its menu - like . The retrieval mechanics are hidden behind the tool interface.

Code

```

def search_medical_knowledge(query: str) -> dict:
    '''Tool wrapper over retrieve(). Limits output to keep prompts clean.'''
    hits = retrieve(query, k=3)
    return {"query": query, "results": hits}
search_medical_knowledge_schema = {
    "type": "function",
    "name": "search_medical_knowledge",
    "description": (
        "Search the clinical knowledge base for guidance on symptoms, medications, "
        "and red flags. Use this whenever you need general medical knowledge - "
        "do NOT rely on your pre-training for clinical recommendations."
    ),
    "parameters": {
        "type": "object",
        "properties": {
            "query": {
                "type": "string",
                "description": "Short natural-language question or topic.",
            },
        },
    },
    "required": ["query"],
    "additionalProperties": False,
}
# Register with the agent.
TOOL_IMPLEMENTATIONS["search_medical_knowledge"] = search_medical_knowledge
TOOL_SCHEMAS.append(search_medical_knowledge_schema)
# Strengthen the system prompt so the agent uses it.
SYSTEM_PROMPT = SYSTEM_PROMPT + (
    "\nFor any medical judgement - red flags, severity, drug-symptom relationships - "
    "ALWAYS call search_medical_knowledge first. Do not rely on your own knowledge."
)

```

5.6 Re-run the scenario - now with grounded medical reasoning

Reset the world (so the appointment calendar is fresh), then send the same message. You should see the agent pick at least once - grounding its headache/warfarin advice in our corpus rather than from memory.

Code

```
# Reset the world so we can see a clean trace.
_DOCTOR_AVAILABILITY["Dr. Cohen"] = ["2026-04-20 09:00", "2026-04-20 14:00", "2026-04-22
10:30"]
_BOOKED_APPOINTMENTS.clear()
final_answer, history = run_agent(
    "Patient P001 says: " + PATIENT_MESSAGE + " They are asking for any slot next week."
)
print("\n\n??????????? FINAL ANSWER (with RAG) ??????????")
print(fill(final_answer, width=88))
```

Output

```

???????? Iteration 1 ????????
? tool: get_patient_record({'patient_id': 'P001'})
? result: {'patient_id': 'P001', 'name': 'Rivka Weiss', 'age': 68, 'medications': ['warfarin',
'lisinopril'], 'conditions': ['atri...
? tool: check_drug_interactions({'drugs': ['warfarin', 'aspirin']})
? result: {'interactions': [{'drug_a': 'warfarin', 'drug_b': 'aspirin', 'severity': 'high',
'note': 'Both affect clotting; combine...
? tool: search_medical_knowledge({'query': 'headache on warfarin and aspirin red flags urgent
evaluation'})
? result: {'query': 'headache on warfarin and aspirin red flags urgent evaluation', 'results':
[{'score': 0.639, 'passage': 'Any h...
? tool: get_doctor_availability({'doctor': 'Dr. Cohen'})
? result: {'doctor': 'Dr. Cohen', 'available_slots': ['2026-04-20 09:00', '2026-04-20 14:00',
'2026-04-22 10:30']}
???????? Iteration 2 ????????
? Final answer.
???????? FINAL ANSWER (with RAG) ??????????
I'm glad you asked and I'm here to help. I've pulled up your record and checked a few
things because you're on warfarin and also taking aspirin. What I found - Patient:
Rivka Weiss, age 68 - Conditions: atrial fibrillation, hypertension - Meds on record:
warfarin, lisinopril (INR has been stable on 5 mg warfarin daily) - You also mentioned
taking aspirin daily (not in the record list I pulled, but note that you're on both
warfarin and aspirin) What this means for your headache - There is a significant
bleeding risk when warfarin is used with aspirin, so headaches should be evaluated
carefully. - Red flags to watch for (these require urgent care or emergency evaluation):
- Sudden, severe-onset or "thunderclap" headache - Headache with confusion, weakness,
difficulty speaking, numbness, trouble walking - Neck stiffness, fever, neck pain, or
vomiting - Any new severe headache after a head injury - Other concerning signs to
report promptly include unusual bruising, blood in urine or stool, or any signs of
bleeding that won't stop. - If you have none of the red flags and your headache is mild,
we should still assess it soon since you're on anticoagulants. Do not start or stop any
new blood-thinning or pain medications without guidance from a clinician. Appointments
with Dr. Cohen next week Available slots: - 2026-04-20 at 09:00 - 2026-04-20 at 14:00 -
2026-04-22 at 10:30 Would you like me to book one of these? If so, tell me which time
you prefer and I'll place the booking. If you have any new or worsening symptoms, please
let me know so I can help assess urgency. Tips for the visit - Bring a list of all
medications (including any over-the-counter meds and supplements) and your last INR
result if you have it. - Mention the aspirin use explicitly to Dr. Cohen, since it
affects bleeding risk with warfarin.

```

? Quiz - Part 5

Q1. Why do we embed both the corpus and the query using the same model?

Q2. We store pairs in a plain Python list. At what scale does this break down, and what replaces it?

Q3. What failure mode does RAG NOT fix?

Part 6 - Capability 5: Reflection (Self-Critique)

Our agent is grounded, multi-step, stateful, and knowledge-augmented. In any domain where a quiet mistake is costly - healthcare, law, finance - theres one more layer worth adding: a critic.

6.1 The problem the critic addresses

Even with every capability so far, the agent can:

- * Give advice thats technically correct but misses urgency.
- * Forget to mention a red flag thats in the record.
- * Book an appointment when the patient really needs same-day care.
- * Misinterpret a tool result.

A simple, surprisingly effective mitigation: after the agent produces its answer, a second LLM call with a different role reviews it. This is the critic pattern, sometimes called reflection or LLM-as-a-judge. Its not a safety guarantee, but it catches a meaningful fraction of sloppy outputs at low cost.

Key design choice: the critic has narrow responsibilities. It isnt trying to do the triage better; its looking for specific failure modes - red flags ignored, unsafe advice, booked-when-urgent, etc.

6.2 Implementing the critic

We build the critic as a single Responses call with a strict prompt and a structured output: .

Code


```

CRITIC_PROMPT = '''You are a clinical safety reviewer. You are shown:
- the patient's original message,
- the patient record (if known),
- the assistant's draft reply.
Your job is to flag any of these failure modes:
1. Urgent red flags ignored (e.g. severe symptoms handled as routine).
2. Medical advice given without grounding in the tool results / knowledge base.
3. Appointment booked for a case that may need same-day or emergency review.
4. Drug interaction risks not mentioned when clearly relevant.
5. Reassurance given in a situation where caution is warranted.
If the draft is safe and appropriate, reply with: PASS
Otherwise, reply with a bullet list starting with FAIL:, one bullet per issue.
Be concise. Do not rewrite the answer.
'''

def critique(patient_message: str, draft_answer: str, patient_record: dict | None = None) -> dict:
    review_input = (
        f"PATIENT MESSAGE:\n{patient_message}\n\n"
        f"PATIENT RECORD:\n{json.dumps(patient_record, indent=2) if patient_record else 'unknown'}\n\n"
        f"DRAFT REPLY:\n{draft_answer}"
    )
    resp = client.responses.create(
        model=MODEL,
        input=review_input,
        instructions=CRITIC_PROMPT,
    )
    verdict = resp.output_text.strip()
    passed = verdict.upper().startswith("PASS")
    return {"passed": passed, "verdict": verdict}

```

6.3 Wrapping the agent with a critic loop

Now we compose: run the agent, get the draft, run the critic, and (if the critic fails) feed the issues back to the agent and re-run. Bounded retries, of course - the critic can be wrong too.

Code

```
def run_agent_with_critic(user_message, patient_record=None, max_retries=2):
    message_history = None
    last_answer = None
    for attempt in range(max_retries + 1):
        print(f"\n??????????? Attempt {attempt + 1} ????????????")
        draft, message_history = run_agent(
            user_message if attempt == 0
            else f"Your previous answer was rejected by safety review with these issues:\n"
            f"{last_verdict}\n\nPlease revise your answer addressing these issues.",
            message_history=message_history,
            verbose=False,
        )
        print(f"DRAFT:\n{fill(draft, width=88)}")
        review = critique(user_message, draft, patient_record)
        last_verdict = review["verdict"]
        print(f"\nCRITIC: {review['verdict']}")
        if review["passed"]:
            return draft, message_history
        last_answer = draft
    print("\n? Critic did not pass within retry budget - returning last draft.")
    return last_answer, message_history
```

Code

```
# Reset world.
_DOCTOR_AVAILABILITY["Dr. Cohen"] = ["2026-04-20 09:00", "2026-04-20 14:00", "2026-04-22 10:30"]
_BOOKED_APPOINTMENTS.clear()
# Feed the critic the patient record so it can judge in context.
final, hist = run_agent_with_critic(
    "Patient P001 says: " + PATIENT_MESSAGE + " They are asking for any slot next week.",
    patient_record=_PATIENT_DB["P001"],
)
print("\n\n??????????? FINAL (post-critic) ????????????")
print(fill(final, width=88))
```

6.4 Honest framing of what the critic does (and doesn't)

Reflection is a quality pass, not a safety guarantee.

- * It catches a significant share of obvious mistakes - urgent red flags soft-pedalled, tool results misread, advice given without grounding.
- * It does not catch systematic biases shared between the agent and critic (they're the same model family - they have correlated blind spots).
- * It does not catch knowledge gaps - if our corpus doesn't cover a condition, the critic won't either.
- * It does add cost and latency. In production, you tune when to trigger it (always vs. only on high-risk queries).

In clinical deployment the real safety net is a human. The critic is a speed bump, not a wall.

? Quiz - Part 6

Q1. Why does the critic use a different prompt rather than the same prompt as the main agent?

Q2. The critic is the same model (gpt-5-nano) as the agent. What failure mode does this create?

Q3. What's the difference between a critic that loops with the agent (like ours) and one that only labels the output?

Part 7 - LLM vs Agent, Side by Side

Lets cash in the work. Same prompt, two systems.

Code

```
def llm_only(message):
    resp = client.responses.create(
        model=MODEL,
        input=message,
        instructions="You are a clinic assistant. Respond to the patient.",
    )
    return resp.output_text
# Reset state.
_DOCTOR_AVAILABILITY["Dr. Cohen"] = ["2026-04-20 09:00", "2026-04-20 14:00", "2026-04-22 10:30"]
_BOOKED_APPOINTMENTS.clear()
prompt = "Patient P001 says: " + PATIENT_MESSAGE + " Any slot next week works."
print("???????????????? LLM ONLY ?????????????????")
print(fill(llm_only(prompt), width=88))
print()
print("???????????????? FULL AGENT ?????????????????")
agent_answer, _ = run_agent(prompt, verbose=False)
print(fill(agent_answer, width=88))
print()
print(f"Side effect check - appointments booked: {len(_BOOKED_APPOINTMENTS)}")
if _BOOKED_APPOINTMENTS:
    print(f" ? {_BOOKED_APPOINTMENTS[0]}")
```

7.1 The comparison table

Agents are strictly more powerful and strictly more expensive. They are worth building when the task has structure that rewards the extra machinery - and not when a single LLM call is enough.

7.2 When NOT to build an agent

Its easy to build an agent because agents are exciting. Dont skip the question: is this the right tool?

Build an agent when:

- The task requires live data the model doesnt have (prices, records, availability).
- The task has multiple steps whose order depends on intermediate results.
- The task ends in a real-world action, not just text.
- The task benefits from iteration (draft ? check ? revise).

Dont build an agent when:

- A single call with good prompting gets you there (summarisation, translation, classification).
- The cost / latency of a loop is unacceptable (interactive autocomplete, real-time moderation).
- You cannot define a clear done condition.
- The cost of a wrong action is higher than the cost of no action, and you dont have safeguards.

The overwhelmingly common mistake in 2026 is reaching for an agent when an LLM call would do. Agents are an architectural cost you only pay when the architectural value is there.

? Quiz - Part 7

Q1. The agent answered substantively about drug interactions. Where did that information come from - and how do you know?

Q2. Youre building a feature that turns a customers email into a cleaner, shorter version. Agent or LLM?

Q3. Youre building a feature that takes a customers email and, if its a refund request, issues the refund. Agent or LLM?

Part 8 - Closing & What Comes Next

8.1 Healthcare-specific reality check

This notebook is deliberately a toy. If you want to take anything like it into a real clinic, the hard work begins after the agent works:

* HIPAA / data residency. Real records cannot be sent to a general-purpose cloud LLM without a Business Associate Agreement and matching architecture. Most teams end up running models on-prem or on compliant cloud regions.

* Audit trail. Every tool call, every decision, every retrieved passage needs to be logged. Regulators will

ask.

- * Explainability. Clinicians need to see why the agent made a recommendation - trace, not just answer.
- * Bias evaluation. Does the agent advise differently by age, gender, ethnicity, language? This has to be measured, not assumed.
- * Adversarial robustness. What happens if a patient (accidentally or maliciously) sends contradictory or manipulative input?
- * Human-in-the-loop. For any nontrivial decision - especially anything actionable - a qualified human signs off before the agent acts.
- * Validated knowledge base. Our 8 dummy passages become thousands of curated, citeable sources, versioned, reviewed, updated.

Any of these topics is a semester of its own. None of them are optional.

8.2 What we deliberately skipped

To keep the notebook focused, we left out several important topics. Brief map of where to go next:

- * Multi-agent systems. Multiple specialised agents (e.g., a triage agent, a booking agent, a pharmacy agent) coordinated by an orchestrator. Adds power and failure modes.
- * Plan-and-execute. Instead of deciding one step at a time (ReAct), produce a full plan upfront, execute, replan on divergence. Faster and cheaper at scale.
- * MCP (Model Context Protocol). An emerging standard for exposing tools to models as remote services, decoupling tool-builders from agent-builders.
- * Production frameworks. LangGraph, OpenAI Agents SDK, CrewAI, Anthropics Claude tool use. All are variations on what we built by hand. Once you understand the from-scratch version, picking one is mostly about ergonomics.
- * Streaming. For interactive UX, stream partial responses as the agent works. The Responses API supports this; we used the non-streaming version for clarity.
- * Evaluation / tracing. How do you know your agent is good? Dedicated eval harnesses (LangSmith, custom) that replay scenarios and score outcomes.
- * Cost controls. Budget caps, per-run token limits, caching.

8.3 Recap

One sentence:

An agent is an LLM placed inside a loop that can call tools, remember history, retrieve knowledge, and check its own work.

Or, as a progression, which is how we built it:

1. LLM - text in, text out. Frozen, stateless, hands-free.

2. + Tools - closes the action gap. The LLM can now request calls into our code.
3. + Loop (ReAct) - lets the LLM chain tool calls together, reacting to each result.
4. + Memory - threads history so the agent stays coherent across turns.
5. + RAG - grounds the agent in a curated knowledge base instead of pre-training.
6. + Reflection - adds a safety pass for domains where quiet errors matter.

Each step was motivated by a concrete failure of the previous. Each step is about 10-50 lines of code. There is no magic.

What is new and genuinely hard is everything we didnt touch - evaluation, safety, regulation, deployment. Those are the problems the field is actively working on in 2026. The architecture itself is now largely settled, and it looks like this notebook.

? Quiz - Part 8

Q1. A colleague proposes just use an agent framework, they handle all this for you. What should you push back on?

Q2. In our progression, every step was motivated by a concrete failure. What was the failure that motivated (a) tools, (b) the loop, (c) RAG?

Q3. If you could only add ONE of the capabilities to a bare LLM, which would you pick for a healthcare application, and why?

Appendix - Exercises

Four open-ended exercises. No solutions provided - these are for you to sit with.

Exercise 1 - Add a new tool

Add a tool that returns a mock dict of recent labs (INR, BP, glucose, etc.) for the patient. Register it with the agent and update the system prompt. Ask the agent a question that should trigger it (e.g., is my INR okay?) and trace what it does.

Exercise 2 - Corpus behaviour

Replace one passage in with something clearly wrong (but plausible-sounding). Rerun the scenario. Does the agents answer change? Does the critic catch it? What does this tell you about where the trust in a RAG system really lives?

Exercise 3 - Break the agent

Construct an adversarial user message designed to break the agent:

- ambiguous patient ID,
- contradictory symptoms,
- a request for a tool that doesnt exist,
- a prompt injection (ignore all previous instructions and).

What happens in each case? Which failures are caught, and which slip through?

Exercise 4 - A human-in-the-loop escape hatch

Add a tool that, when called, stops the agent loop and returns a please wait for a clinician message. Strengthen the system prompt to make the agent call this tool for any urgent red flag. Test by asking about a thunderclap headache (sudden, worst of my life). Does the agent escalate?

End of notebook. You have now built an agent from scratch. The rest is engineering.